13. 실제 예제를 통한 전체 과정 추적

개요

이 문서는 간단한 Cairo 프로그램을 선정하여 전체 컴파일 파이프라인을 단계별로 추적하고 분석합니다. 피보나치 수열을 계산하는 재귀 함수를 예제로 사용하여 Cairo 소스 코드부터 최종 CASM 코드까지의 변환 과정을 상세히 살펴봅니다.

선정된 예제 프로그램

원본 Cairo 코드

```
// Calculates fib...

pub fn fib(a: felt252, b: felt252, n: felt252) -> felt252 {

match n {

0 => a,

_ => fib(b, a + b, n - 1),

}

}
```

이 함수는 다음과 같은 특징을 가집니다:

- **재귀 함수**: 자기 자신을 호출하는 구조
- **패턴 매칭**: `match` 표현식 사용
- **산술 연산**: 덧셈과 뺄셈 연산

- **조건부 분기**: 기본 케이스와 재귀 케이스

컴파일 단계별 분석

1단계: 소스 코드 파싱 (Cairo → AST)

토큰화 과정

Cairo 파서는 먼저 소스 코드를 토큰으로 분해합니다:

```
PUB, FN, IDENTIFIER("fib"), LPAREN,
IDENTIFIER("a"), COLON, IDENTIFIER("felt252"), COMMA,
IDENTIFIER("b"), COLON, IDENTIFIER("felt252"), COMMA,
IDENTIFIER("n"), COLON, IDENTIFIER("felt252"), RPAREN,
ARROW, IDENTIFIER("felt252"), LBRACE,
MATCH, IDENTIFIER("n"), LBRACE,
LITERAL(0), ARROW, IDENTIFIER("a"), COMMA,
UNDERSCORE, ARROW, IDENTIFIER("fib"), LPAREN, ...
```

AST 구조

파싱된 결과는 다음과 같은 AST 구조를 가집니다:

```
FunctionDeclaration {
visibility: Public,
name: "fib",
parameters: [
```

```
Parameter { name: "a", type: "felt252" },

Parameter { name: "b", type: "felt252" },

Parameter { name: "n", type: "felt252" }

],

return_type: "felt252",

body: MatchExpression {

expr: Identifier("n"),

arms: [

MatchArm {

pattern: Literal(0),

expression: Identifier("a")

},

MatchArm {

pattern: Wildcard,

expression: FunctionCall {

name: "fib",

args: [

Identifier("b"),

BinaryOp {

left: Identifier("a"),

op: Add,

right: Identifier("b")

},
```

```
BinaryOp {
left: Identifier("n"),
op: Sub,
right: Literal(1)
}
]
}
}
]
}
}
```

2단계: 의미 분석 (AST → HIR)

타입 체킹

의미 분석 단계에서는 다음과 같은 검증이 수행됩니다:

1. **타입 일치성 검사**:
   - 모든 매개변수가 `felt252` 타입
   - 반환 타입이 `felt252` 와 일치
   - 산술 연산의 피연산자 타입 검증
2. **스코프 해결**:
   - 변수 `a`, `b`, `n` 의 바인딩 확인
   - 재귀 호출에서 함수 이름 해결
3. **패턴 매칭 검증**:

- 모든 가능한 케이스가 처리되는지 확인
- 패턴의 타입 일치성 검사

HIR 표현

의미 분석 후 생성되는 HIR은 타입 정보가 추가된 형태입니다:

```
Function {
id: FunctionId(0),
signature: Signature {
params: [felt252, felt252, felt252],
return_type: felt252
},
body: Match {
scrutinee: Variable(n: felt252),
arms: [
Arm {
pattern: Literal(0: felt252),
body: Variable(a: felt252)
},
Arm {
pattern: Otherwise,
body: Call {
function: FunctionId(0), // 재귀 호출
args: [
```

```
Variable(b: felt252),

Add(Variable(a: felt252), Variable(b: felt252)),

Sub(Variable(n: felt252), Literal(1: felt252))

]

}

}

]

}

}
```

3단계: Lowering (HIR → MIR)

패턴 매칭 해제

Lowering 단계에서는 고수준 구조를 더 단순한 형태로 변환합니다:

```
Function fib(a: felt252, b: felt252, n: felt252) -> felt252 {

let temp_0 = felt252_is_zero(n);

match temp_0 {

IsZeroResult::Zero => {

return a;

},

IsZeroResult::NonZero(nz_n) => {

let temp_1 = felt252_add(a, b);

let temp_2 = felt252_sub(n, 1);
```

```
let result = fib(b, temp_1, temp_2);

return result;

}

}

}
```

제어 흐름 단순화

- `match` 표현식이 조건부 분기로 변환
- 산술 연산이 명시적인 함수 호출로 변환
- 임시 변수 도입으로 복잡한 표현식 분해

4단계: Sierra 중간표현 생성

Sierra 타입 선언

```
type [1395713] = felt252 [storable: true, drop: true, dup: true, zero_sized: fal

type [1395715] = RangeCheck [storable: true, drop: false, dup: false, zero_sized

type [1395716] = GasBuiltin [storable: true, drop: false, dup: false, zero_sized

type [1395717] = NonZero<[1395713]> [storable: true, drop: true, dup: true, zero

type [1395718] = Struct<ut@Tuple, [1395713]> [storable: true, drop: true, dup: t
```

Sierra 라이브러리 함수

```
libfunc [1399827] = felt252_is_zero;

libfunc [1399823] = felt252_add;
```

```
libfunc [1399809] = felt252_sub;

libfunc [1399822] = function_call<user@[1403909]>;

libfunc [1399828] = withdraw_gas;

libfunc [1399824] = redeposit_gas;
```

Sierra 함수 본체

```
F0: // fib 함수

[1399829]() -> (); // disable_ap_tracking

[1399828]([0], [1]) { fallthrough([5], [6]) F0_B1([7], [8]) }; // withdraw_gas

[1399830]() -> (); // branch_align

[1399831]([4]) -> ([4], [9]); // dup n

[1399827]([9]) { fallthrough() F0_B0([10]) }; // felt252_is_zero(n)

[1399830]() -> (); // branch_align

[1399832]([4]) -> (); // drop n

[1399832]([3]) -> (); // drop b

[1399824]([6]) -> ([11]); // redeposit_gas

[1399826]([2]) -> ([12]); // struct_construct(a)

[1399825]([12]) -> ([13]); // enum_init (success case)

return([5], [11], [13]); // return a


F0_B0: // 재귀 케이스

[1399830]() -> (); // branch_align
```

```
[1399833]([10]) -> (); // drop NonZero

[1399824]([6]) -> ([14]); // redeposit_gas

[1399831]([3]) -> ([3], [15]); // dup b

[1399823]([2], [15]) -> ([16]); // felt252_add(a, b)

[1399812]() -> ([17]); // const_as_immediate(1)

[1399809]([4], [17]) -> ([18]); // felt252_sub(n, 1)

[1399822]([5], [14], [3], [16], [18]) -> ([19], [20], [21]); // recursive call

return([19], [20], [21]); // return result
```

5단계: Sierra 최적화

가스 계산 최적화

Sierra 코드에서 가스 관련 최적화가 적용됩니다:

- `withdraw_gas` : 함수 시작 시 가스 소모
- `redeposit_gas` : 미사용 가스 반환
- 가스 효율적인 명령어 선택

AP 변화 추적

- `disable_ap_tracking` : AP 추적 비활성화로 성능 향상
- `branch_align` : 분기 정렬로 예측 가능한 메모리 접근

6단계: CASM 코드 생성

최종 CASM 출력

```
%{ memory[ap + 0] = 1170 <= memory[fp + -6] %} // 가스 체크
```

```
jmp rel 7 if [ap + 0] != 0, ap++; // 가스 부족 시 분기

[ap + 0] = [fp + -6] + 340282366920938463463374607431768210286, ap++; // 가스 차감

[ap + -1] = [[fp + -7] + 0]; // 가스 상태 저장

jmp rel 27; // 에러 처리로 점프


[fp + -6] = [ap + 0] + 1170, ap++; // 가스 복원

[ap + -1] = [[fp + -7] + 0]; // 가스 상태 저장

jmp rel 12 if [fp + -3] != 0; // n이 0이 아니면 재귀 케이스로


// 기본 케이스 (n == 0)

[ap + 0] = [fp + -7] + 1, ap++; // 가스 포인터 증가

[ap + 0] = [ap + -2] + 2070, ap++; // 가스 계산

[ap + 0] = 0, ap++; // 성공 태그

[ap + 0] = 0, ap++; // 패닉 구조체

[ap + 0] = [fp + -5], ap++; // 결과값 (a)

ret; // 반환


// 재귀 케이스 (n != 0)

[ap + 0] = [fp + -7] + 1, ap++; // 가스 포인터 증가

[ap + 0] = [ap + -2], ap++; // 가스 복사

[ap + 0] = [fp + -4], ap++; // b 값

[ap + 0] = [fp + -5] + [fp + -4], ap++; // a + b 계산

[fp + -3] = [ap + 0] + 1, ap++; // n - 1 계산
```

```
call rel -29; // 재귀 호출

ret; // 반환
```

데이터 변환 추적

메모리 레이아웃 변화

1. 함수 호출 시작

```
fp-7: RangeCheck 포인터

fp-6: Gas builtin 포인터

fp-5: 매개변수 a (felt252)

fp-4: 매개변수 b (felt252)

fp-3: 매개변수 n (felt252)
```

2. 조건 검사 후

```
ap+0: n == 0 검사 결과 (boolean)

ap+1: 가스 차감 후 상태
```

3. 재귀 호출 준비

```
ap+0: 새로운 가스 포인터

ap+1: 가스 상태

ap+2: 새로운 a 값 (이전 b)
```

```
ap+3: 새로운 b 값 (a + b)

ap+4: 새로운 n 값 (n - 1)
```

타입 변환 과정

1. **Cairo 타입 → Sierra 타입**:

- `felt252 → type [1395713] = felt252`
- 함수 시그니처 → Sierra 함수 선언

2. **Sierra 타입 → CASM 메모리**:

- `felt252` → 메모리 셀 (252비트 정수)
- 구조체 → 연속된 메모리 영역
- 열거형 → 태그 + 데이터

성능 분석

가스 소모 패턴

기본 케이스 (n=0)

- 가스 체크: ~10 단위
- 조건 검사: ~5 단위
- 반환: ~3 단위
- **총합**: ~18 가스 단위

재귀 케이스 (n>0)

- 가스 체크: ~10 단위
- 조건 검사: ~5 단위

- 산술 연산: ~8 단위 (덧셈 + 뺄셈)
- 재귀 호출: ~15 단위 + 하위 호출 비용
- **총합**: ~38 + 재귀 비용

전체 복잡도

- 시간 복잡도: O(n) - 선형 재귀
- 가스 복잡도: O(n) - 각 재귀 레벨당 일정한 가스
- 메모리 복잡도: O(n) - 호출 스택 깊이

최적화 기회

1. **꼬리 재귀 최적화**: 현재 구현은 꼬리 재귀가 아님
2. **반복문 변환**: 재귀를 반복문으로 변환 가능
3. **메모이제이션**: 중복 계산 방지 (이 예제에서는 해당 없음)

컴파일러 최적화 분석

Sierra 레벨 최적화

1. **가스 최적화**:
- 불필요한 가스 체크 제거
- 가스 재예치 최적화
2. **AP 추적 최적화**:
- `disable_ap_tracking` 으로 성능 향상
- 예측 가능한 메모리 접근 패턴
3. **분기 최적화**:

- `branch_align` 으로 분기 예측 향상
- 조건부 점프 최적화

CASM 레벨 최적화

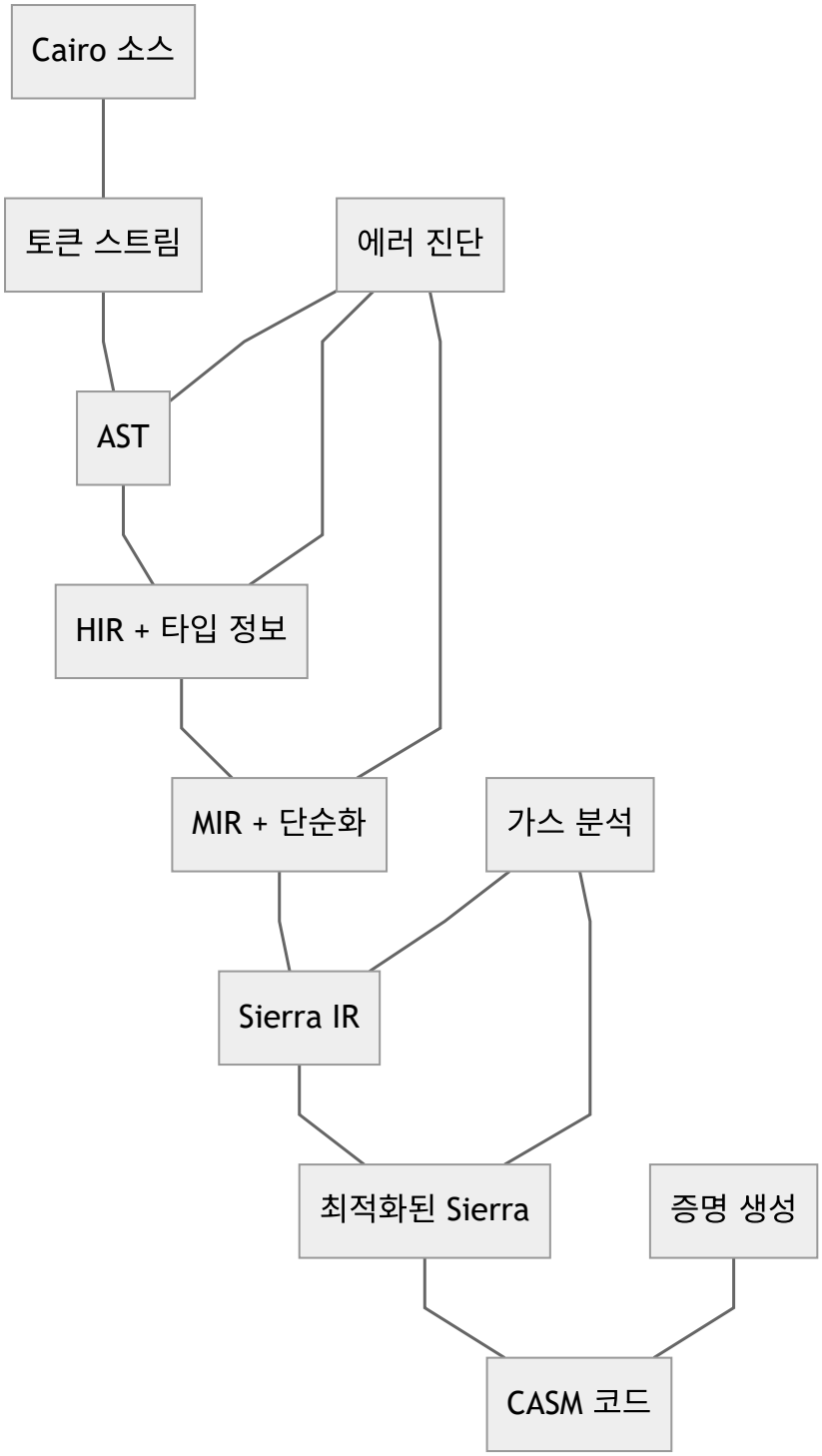
1. **레지스터 할당**:
- 효율적인 메모리 사용
- 중간값 최소화
2. **명령어 스케줄링**:
- 파이프라인 최적화
- 메모리 접근 패턴 최적화

전체 파이프라인 통합 이해

컴파일 단계별 목적

1. **파싱**: 구문 구조 이해 및 검증
2. **의미 분석**: 타입 안전성 및 의미 검증
3. **Lowering**: 고수준 구조의 단순화
4. **Sierra 생성**: 증명 가능한 중간 표현
5. **최적화**: 성능 및 가스 효율성 향상
6. **CASM 생성**: 실행 가능한 어셈블리 코드

각 단계의 상호작용



핵심 설계 원칙

1. **점진적 변환**: 각 단계는 이전 단계를 기반으로 구축
2. **타입 안전성**: 모든 단계에서 타입 정보 보존
3. **증명 가능성**: Sierra를 통한 실행 추적 가능
4. **최적화 가능성**: 각 레벨에서 적절한 최적화 적용

결론

이 피보나치 예제를 통해 Cairo 컴파일러의 전체 파이프라인을 살펴본 결과:

1. **구조적 변환**: 고수준 Cairo 코드가 단계적으로 저수준 CASM으로 변환

2. **타입 보존**: 전 과정에서 타입 안전성 유지

3. **최적화 적용**: 각 레벨에서 적절한 최적화 기법 사용

4. **증명 지원**: Sierra를 통한 실행 증명 생성 가능

이러한 체계적인 컴파일 과정을 통해 Cairo는 안전하고 효율적이며 증명 가능한 코드를 생성할 수 있습니다.